Operational Semantics

Full Rule Set with Await-Local Timers

1 What changes

This version retains the draft's machine $\Sigma = (M, P, C, S, K, T)$, small-step expressions, frame stack, async thread pool, rule names, and clock-scaling convention. There is no new energy counter, duration-labelled transition, power model, or scheduler. Resource consumption is recorded by the existing transition label q and summed over a run.

The local corrections are: charge the operations already charged by the typing rules; distinguish computational steps from free frame bookkeeping; use $\to^*$ when a premise requires complete expression evaluation; retain a pool entry while its async job runs; add an await-waiting rule; and repair the continuation frames needed for sequence, return, and completed branches. The await timer starts only when await becomes the current command. Timeout implementation remains outside scope. The ambient cost term is not command overhead: it represents peripherals supplied by a function call site.

2 Machine, costs, and conventions

$$\Sigma = (M, P, C, S, K, T), \qquad K \in \mathbb{Q}_{>0}, \quad T \in \mathbb{Q}_{\geq 0}.$$

M, P, C, S, K, T have their original meanings. T is rational because the draft increments it by the rational quantity K. Write $X = (M, P, C, S, K)$.

$$\begin{aligned}
P &::= tid \mapsto (p, c), P \mid tid \mapsto \mathsf{done}(p, \mathsf{skip}), P \mid \cdot, \\
F &::= (tid : p, c) \mid (tid : p, [\,]; c) \mid (tid : p, \mathsf{let}\ x = [\,]) \mid (tid : p, \mathsf{endif}), \\
C &::= F :: C \mid \cdot.
\end{aligned}$$

The return frame is the draft's $\mathsf{let}\ x = [\,]; c$ frame with the following command c held by the existing sequence continuation. This avoids a free continuation variable in E-Call.

Let $b(p, s, K)$ be the peripheral resource cost per elapsed-time unit. The actual background cost of one computational step is

$$\mathrm{bg}(S, K) = K \sum_{p \in \mathrm{dom}(S)} b(p, S(p), K).$$

At runtime S includes all peripherals, so this sum includes both the known peripherals represented by Δ and the ambient peripherals represented by $\mathrm{cost}(\cdot)$ after function instantiation. There is no extra command overhead term. Software instruction energy is not separately modelled here. Transition and clock-update costs remain explicit, as in the draft.

The draft's abstract clock convention is retained: a computational step at K advances T by K. The functions b and cost use consistent resource units; this does not reinterpret K as physical frequency.

Handles are hygienically bound names, so the fresh name introduced by $\mathsf{let}\ tid = \mathsf{async}(p)\{c\}$ is the same handle used by its awaits. Captured peripherals are inaccessible to other software threads, as required by the typing context. Function and async locals are freshened to prevent accidental interference through M.

3 Expressions: unchanged

$$\frac{M(x) = v}{M, S \vdash x \to v} \text{ E-Var} \qquad \frac{M, S \vdash e_1 \to e_1'}{M, S \vdash e_1 \text{ binop } e_2 \to e_1' \text{ binop } e_2} \text{ E-BinopStep1}$$

$$\frac{M, S \vdash e_2 \to e_2'}{M, S \vdash v_1 \text{ binop } e_2 \to v_1 \text{ binop } e_2'} \text{ E-BinopStep2}$$

$$\frac{v_1 \text{ binop } v_2 = v}{M, S \vdash v_1 \text{ binop } v_2 \to v} \text{ E-BinopFin}$$

Below, $M, S \vdash e \to^* v$ is the reflexive transitive closure of these rules. Expression reductions are free in this version, matching the current typing rule that only sums operand costs. The enclosing computational command pays background cost. Nonzero expression costs would require matching labels in both the expression semantics and expression typing rules.

4 Sequential commands

4.1 Sequence and frame completion

Use the sequence-hole frame already declared in the draft. These are free administrative steps, so T-Seq can simply add subcommand costs.

$$\frac{}{(M, P, (tid : p, c_1; c_2) :: C, S, K) \rightsquigarrow_0^{\text{adm}} (M, P, (tid : p, c_1) :: (tid : p, [\,]; c_2) :: C, S, K)} \text{ E-Seq}$$

$$\frac{}{(M, P, (tid : p, \mathsf{skip}) :: (tid : p, [\,]; c) :: C, S, K) \rightsquigarrow_0^{\text{adm}} (M, P, (tid : p, c) :: C, S, K)} \text{ E-SeqDone}$$

$$\frac{}{(M, P, (tid : p, \mathsf{skip}) :: (tid : p, \mathsf{endif}) :: C, S, K) \rightsquigarrow_0^{\text{adm}} (M, P, (tid : p, \mathsf{skip}) :: C, S, K)} \text{ E-EndIf}$$

4.2 Assignment and branching

$$\frac{M, S \vdash e \to^* v \qquad q = \text{bg}(S, K)}{(M, P, (tid : p, \mathsf{let}\ x = e) :: C, S, K) \rightsquigarrow_q (M[x \mapsto v], P, (tid : p, \mathsf{skip}) :: C, S, K)} \text{ E-Assign}$$

$$\frac{M, S \vdash e \to^* \mathsf{true} \qquad q = \text{bg}(S, K)}{\begin{array}{c}(M, P, (tid : p, \mathsf{if}\ e\ \mathsf{then}\ c_1\ \mathsf{else}\ c_2) :: C, S, K) \rightsquigarrow_q \\ (M, P, (tid : p, c_1) :: (tid : p, \mathsf{endif}) :: C, S, K)\end{array}} \text{ E-IfTrue}$$

$$\frac{M, S \vdash e \to^* \mathsf{false} \qquad q = \text{bg}(S, K)}{\begin{array}{c}(M, P, (tid : p, \mathsf{if}\ e\ \mathsf{then}\ c_1\ \mathsf{else}\ c_2) :: C, S, K) \rightsquigarrow_q \\ (M, P, (tid : p, c_2) :: (tid : p, \mathsf{endif}) :: C, S, K)\end{array}} \text{ E-IfFalse}$$

4.3 Functions

Retain the draft's stored function bodies. Before entering the body, instantiate the declared ordinary and peripheral parameters with the actual arguments capture-avoidingly, and freshen local binders. Write the resulting body as $c'; \mathsf{ret} := e_r$. Parameter names can be retained as metadata with the stored body; this is binding information rather than a new machine component.

$$\frac{M, S \vdash e \to^* v \qquad \text{body}(M(f), v, p) = c'; \mathsf{ret} := e_r}{\begin{array}{c}(M, P, (tid : r, \mathsf{let}\ x = f(e, p)) :: C, S, K) \rightsquigarrow_0^{\text{adm}} \\ (M, P, (tid : r, c'; \mathsf{ret} := e_r) :: (tid : r, \mathsf{let}\ x = [\,]) :: C, S, K)\end{array}} \text{ E-Call}$$

$$\frac{M, S \vdash e \to^* v}{(M, P, (tid : r, \mathsf{ret} := e) :: (tid : r, \mathsf{let}\ x = [\,]) :: C, S, K) \rightsquigarrow_0^{\mathrm{adm}} (M[x \mapsto v], P, (tid : r, \mathsf{skip}) :: C, S, K)} \quad \text{E-Ret}$$

Call entry and return-value copying remain free, as in the original semantics. A function summary consists of the charges in its computational body. An additional nonnegative return allowance in a typing rule is conservative but is not necessary for this administrative return convention.

5 Peripheral and clock commands

$$\frac{S(x) = st' \qquad st' \le st \qquad q = \mathrm{cost}(st', st)(K) + \mathrm{bg}(S \setminus x, K)}{(M, P, (tid : p, \mathsf{transition}(x, st)) :: C, S, K) \rightsquigarrow_q (M, P, (tid : p, \mathsf{skip}) :: C, S[x \mapsto st], K)} \quad \text{E-Transition}$$

$$\frac{S(x) = st' \qquad (st' \hookrightarrow st) \in x(D) \qquad q = \mathrm{cost}(st', st)(K) + \mathrm{bg}(S \setminus x, K)}{(M, P, (tid : p, \mathsf{downgrade}(x, st)) :: C, S, K) \rightsquigarrow_q (M, P, (tid : p, \mathsf{skip}) :: C, S[x \mapsto st], K)} \quad \text{E-Downgrade}$$

$$\frac{K' = f \qquad q = \mathrm{cost}_{\mathrm{clock}}(K, K') + \mathrm{bg}(S, K)}{(M, P, (tid : p, \mathsf{clock}\ f) :: C, S, K) \rightsquigarrow_q (M, P, (tid : p, \mathsf{skip}) :: C, S, K')} \quad \text{E-Clock}$$

Let $m = \mathsf{match}\ p\ \mathsf{with}\ st_1 \to c_1 | \cdots | st_n \to c_n$.

$$\frac{S(p) = st_i \qquad q = \mathrm{bg}(S, K)}{(M, P, (tid : r, m) :: C, S, K) \rightsquigarrow_q (M, P, (tid : r, c_i) :: C, S, K)} \quad \text{E-Match-i}$$

These rules charge exactly the corresponding command, transition, and background terms. Concrete runtime states replace the typing rules' maxima over sets of states.

6 Async and await

6.1 Spawn, preemption, and completion

Let $\mathrm{threads}(C)$ return the thread identifiers appearing in C. The async body has fresh local names and cannot modify parent-local variables.

$$\frac{tid \notin \mathrm{dom}(P) \cup \mathrm{threads}(C) \qquad \mathrm{unstable}(S(p)) \qquad q = C_{\mathrm{spawn}}(p, S(p), K) + \mathrm{bg}(S \setminus p, K)}{(M, P, (tid' : r, \mathsf{let}\ tid = \mathsf{async}(p)\{c_a\}) :: C, S, K) \rightsquigarrow_q (M, P[tid \mapsto (p, c_a)], (tid' : r, \mathsf{skip}) :: C, S, K)} \quad \text{E-Async}$$

To match the spawn typing rule, C_{spawn} includes the captured peripheral's own resource use during spawning. Other peripherals are charged by $\mathrm{bg}(S \setminus p, K)$. If the declared spawn cost excludes that own resource use, add it to both the operational and typing costs.

$$\frac{P(tid) = (p, c_a) \qquad tid \notin \mathrm{threads}(C)}{(M, P, C, S, K) \rightsquigarrow_0^{\mathrm{adm}} (M, P, (tid : p, c_a) :: C, S, K)} \quad \text{E-AsyncPreempt}$$

Unlike the original rule, this keeps the pool entry while the job runs. The extra premise prevents the same job from being dispatched twice.

$$\frac{tid \neq \mathsf{main} \qquad \mathrm{root}(tid, C') \qquad \mathrm{stable}(S(p))}{(M, P, (tid : p, \mathsf{skip}) :: C', S, K) \rightsquigarrow_0^{\mathrm{adm}} (M, P[tid \mapsto \mathsf{done}(p, \mathsf{skip})], C', S, K)} \quad \text{E-AsyncDone}$$

$\mathrm{root}(tid, C')$ means that C' is empty or its top frame belongs to another thread. This prevents completion at an intermediate skip.

6.2 Await and waiting

Abbreviate the source command and its active runtime form by

$$w(tid, n, c) = \mathsf{await}\ tid\ \mathsf{timeout}[n]\{c\}, \qquad w^{\bullet}(tid, n, c) = \widehat{\mathsf{await}}\ tid\ \mathsf{timeout}[n]\{c\}.$$

$$\frac{}{(M, P, (tid' : r, w(tid, n, c_{\mathrm{fb}})) :: C, S, K) \rightsquigarrow_0^{\mathrm{adm}} (M, P, (tid' : r, w^{\bullet}(tid, n, c_{\mathrm{fb}})) :: C, S, K)} \quad \text{E-AwaitStart}$$

Apply E-AwaitStart before dispatching an async job once await is the current command. It does not charge resource or advance time.

$$\frac{P(tid) = \mathsf{done}(p, \mathsf{skip})}{(M, P, (tid' : r, w^{\bullet}(tid, n, c_{\mathrm{fb}})) :: C, S, K) \rightsquigarrow_0^{\mathrm{adm}} (M, P \setminus tid, (tid' : r, \mathsf{skip}) :: C, S, K)} \quad \text{E-Await}$$

$$\frac{P(tid) = (p, c_a) \qquad n > 0 \qquad q = \mathrm{bg}(S, K)}{(M, P, (tid' : r, w^{\bullet}(tid, n, c_{\mathrm{fb}})) :: C, S, K) \rightsquigarrow_q (M, P, (tid' : r, w^{\bullet}(tid, n, c_{\mathrm{fb}})) :: C, S, K)} \quad \text{E-AwaitWait}$$

The global step decrements the timer. This rule models an idle waiting step. If an async command executes instead, that command's rule charges its resource use and background activity. Do not take a waiting step simultaneously with that computational step or charge its background cost twice.

6.3 Timeout: retain the draft's abstract handoff

The original timeout rule is retained for a job that is not currently running:

$$\frac{P(tid) = (p, c_a) \qquad n \le 0 \qquad tid \notin \mathrm{threads}(C)}{(M, P, (tid' : r, w^{\bullet}(tid, n, c_{\mathrm{fb}})) :: C, S, K) \rightsquigarrow_0^{\mathrm{adm}} (M, P \setminus tid, (tid' : r, c_{\mathrm{fb}}) :: C, S, K)} \quad \text{E-Await-Timeout}$$

For a running job, the same abstract handoff must also remove its frames:

$$\frac{\begin{array}{c} P(tid) = (p, c_a) \qquad n \le 0 \\ C = C_a +\!\!+ ((tid' : r, w^{\bullet}(tid, n, c_{\mathrm{fb}})) :: C_0) \\ C_a \text{ consists only of } tid \text{ frames} \end{array}}{(M, P, C, S, K) \rightsquigarrow_0^{\mathrm{adm}} (M, P \setminus tid, (tid' : r, c_{\mathrm{fb}}) :: C_0, S, K)} \quad \text{E-Timeout-Running}$$

This merely states the draft's exclusive handoff assumption; it does not model its implementation. It preserves the actual peripheral state, rather than resetting it. If timeout only notifies the parent and leaves the job active, the current fallback typing rule cannot give the parent access to that peripheral. This local running-timeout rule assumes no unrelated async job's frames sit above the waiting parent, for example in programs with one outstanding async operation at a time. An unrestricted multi-job scheduler needs a more general timeout-handler rule; it is not silently covered here.

6.4 Hardware steps

Keep the original state-update shape and pool premise:

$$\frac{P(tid) = (p, c_a) \qquad st' \in \mathrm{HWReach}(S(p))}{(M, P, C, S, K) \rightsquigarrow_0^{\mathrm{adm}} (M, P, C, S[p \mapsto st'], K)} \quad \text{E-HWStep}$$

Here hardware events are instantaneous state changes; their resource use is represented by background costs during computational timesteps. This is the smallest accounting convention compatible with the existing background-only C_{unstable} term. If the original nonzero hardware-transition label is retained instead, its total must also be added to the typing bound. Exclude infinite zero-time hardware runs, for example by an acyclic hardware-only graph. Hardware reachability and the typing state's possible set must agree, including stable endpoints when applicable.

7 Global steps, timers, and total cost

Computational local steps use $\rightsquigarrow_q$. Administrative local steps use $\rightsquigarrow_0^{\text{adm}}$. This distinction is by rule, not by whether its cost happens to be zero. Computational rules are E-Assign, E-IfTrue, E-IfFalse, E-Transition, E-Downgrade, E-Clock, E-Match-i, E-Async, and E-AwaitWait. All other local rules are administrative.

$$\frac{(M,P,C,S,K) \rightsquigarrow_q (M',P',C',S',K') \qquad C'' = \text{drain}_K(C')}{(M,P,C,S,K,T) \xrightarrow{q} (M',P',C'',S',K',T+K)} \quad \text{E-Step}$$

$$\frac{(M,P,C,S,K) \rightsquigarrow_0^{\text{adm}} (M',P',C',S',K')}{(M,P,C,S,K,T) \xrightarrow{0} (M',P',C',S',K',T)} \quad \text{E-Admin}$$

Here the old clock K applies to the current step; K' applies to later steps. Define draining on frames and stacks:

$$\begin{aligned} D_d((tid : p, w^\bullet(h,n,c))) &= (tid : p, w^\bullet(h, \max(0, n-d), c)), \\ D_d(F) &= F \quad \text{for every other frame,} \\ \text{drain}_d(\cdot) &= \cdot, \\ \text{drain}_d(F :: C) &= D_d(F) :: \text{drain}_d(C). \end{aligned}$$

Thus only started timers run. An active await under async frames continues counting down; an await still inside unexecuted command syntax does not.

Scheduling conventions. Process frame-completion and async-completion bookkeeping, then any expired active await, before further computational execution. Start a current source await before async dispatch. Completion already recorded in P is handled by E-Await. These conventions include expiry of an await underneath a running async job under the stack restriction stated above. Scheduler overhead is administrative in this model.

Cost of a run. No extra accumulator is necessary:

$$\Sigma_0 \xrightarrow{q_1} \Sigma_1 \xrightarrow{q_2} \cdots \xrightarrow{q_m} \Sigma_m, \qquad Q_{\text{run}} = \sum_{i=1}^{m} q_i.$$

8 Declarations

Keep the original declaration judgments. Peripheral metadata remains fixed; S maps a peripheral to its *current state*, as the command rules expect.

$$\frac{S' = S[id \mapsto s_{\text{curr}}]}{(M,S,id\{\langle s_0,\ldots,s_n\rangle, s_{\text{curr}}, <, D\}) \longrightarrow (M,S')} \quad \text{E-PeriphDecl}$$

$$\frac{M' = M[f \mapsto \{c; \mathsf{ret} := e\}]}{\begin{array}{c}(M,S,\mathsf{fn}\ f(x,p) : (\tau_1,\vec{s}_1) \to (\tau_2,\vec{s}_2)\{c; \mathsf{ret} := e\}) \\ \longrightarrow (M',S)\end{array}} \quad \text{E-FunDecl}$$

The stored body retains its formal-parameter binding information for body in E-Call. Declaration processing is not charged to program execution, as in the original draft.

9 Correspondence with the timeout typing bound

At a call site, instantiate the ambient cost using the other runtime peripherals. Interpret the user's expression

$$B = C_{\text{unstable}}(p) + \text{cost}(\Delta \setminus p) + \text{cost}(\cdot)$$

as a bound on total resource use per elapsed-time unit while await is active. The required local property is

$$q_{\text{step}} \leq KB$$

for every computational step while the timer is active, over all reachable states and clocks. This includes explicit software-transition and clock-change costs if those commands are permitted during the interval. If state-cost annotations do not cover those explicit costs, either enlarge the bound or restrict such commands. No separate partial-async-work term is needed when this property holds. $\text{cost}(\cdot)$ is ambient peripheral cost throughout; it is not CPU overhead.

With $K \leq K_{\max}$ and expiry checked between computational steps, the waiting interval is bounded by $n + K_{\max}$, giving the conservative bound

$$Q_{\text{await}} \leq (n + K_{\max})B + Q_{\text{fb}}.$$

The fallback allowance may be unused on successful completion. The tighter $nB + Q_{\text{fb}}$ requires exact deadline interruption or a different normalization that removes final-step overshoot; neither is silently assumed by these atomic-step rules. Steps before E-AwaitStart lie outside this interval and must be accounted for by the surrounding async analysis.

Fallback state and handoff. Typecheck fallback from all states reachable at expiry, not just the state at spawn. Timeout retains the actual S and transfers exclusive access to the fallback under the draft's abstract handoff assumption. Its implementation is not specified. If timeout does not relinquish the job's access, the peripheral must remain captured instead.

Other typing conditions. Branch joins, call preconditions, function-cost instantiation, and hardware reachability must preserve all possible runtime states and clocks. Hardware closure must include stable endpoints when reachable. These are typing-side obligations, not reasons to add machine components. A complete execution must resolve every spawned job by completion or abstract handoff. The target resource theorem is $Q_{\text{run}} \leq Q_{\text{typing}}$; the rules here specify the machine and its costs, rather than asserting a completed proof.